

JAWABAN TECHNICAL TEST

Backend Developer — PT Serasi Autoraya (SERA)

NestJS · TypeScript · PostgreSQL · Redis · Bull Queue

Kandidat	Bintang Wijaya	Tanggal
Email	bintangwijaya18@gmail.com	Juni 2026
GitHub	github.com/rhecustein/sera-test	Demo
Stack	NestJS + TypeScript + PostgreSQL + Redis + Bull	sera.bintanx.com

Section 1 — Logic & Problem Solving

Soal 1 — Marble Problem

i 8 kelereng: 7 berbobot 100g, 1 berbobot 125g. Temukan yang terberat menggunakan timbangan neraca dua lengan maksimal 2 kali penimbangan.

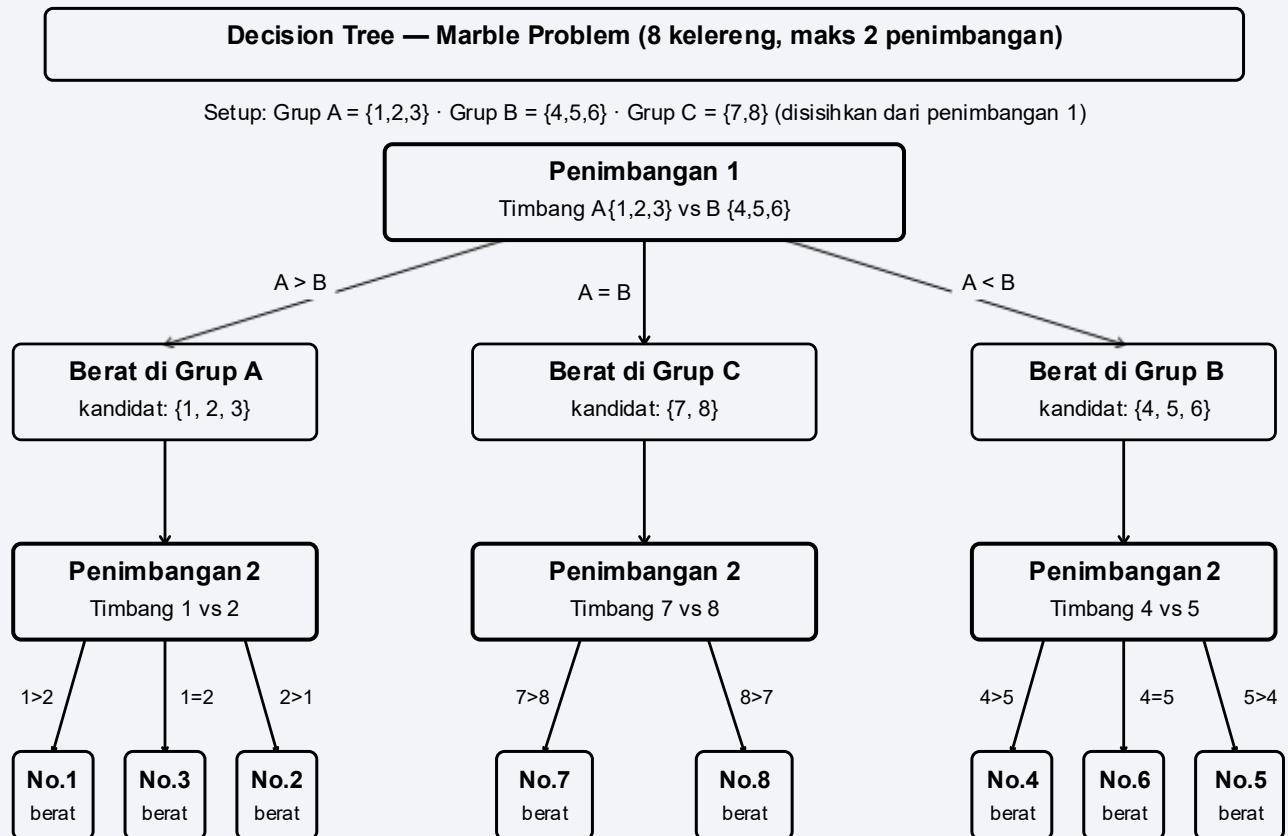
Langkah Algoritma

Pembagian awal: bagi 8 kelereng menjadi 3 grup secara asimetris:

- Grup A: kelereng nomor 1, 2, 3 (3 kelereng)
- Grup B: kelereng nomor 4, 5, 6 (3 kelereng)
- Grup C: kelereng nomor 7, 8 (2 kelereng — disisihkan)

Penimbangan 1 — Timbang Grup A vs Grup B:

Hasil	Kesimpulan	Tindak lanjut ke Penimbangan 2
$A > B$	Berat ada di Grup A	Timbang A1 vs A2 → A1 lebih berat: A1. A2 lebih berat: A2. Seimbang: A3
$A < B$	Berat ada di Grup B	Timbang B1 vs B2 → B1 lebih berat: B1. B2 lebih berat: B2. Seimbang: B3
$A = B$	Berat ada di Grup C	Timbang C1 vs C2 → yang lebih berat itulah jawabannya



Reasoning

Timbangan neraca menghasilkan 3 output: kiri berat / kanan berat / seimbang. Membagi 8 kelereng menjadi 3 grup (3-3-2) memaksimalkan informasi per penimbangan. Jika $A = B$, kelereng berat pasti ada di C tanpa perlu ditimbang di ronde 1. Penimbangan 2: timbang 2 dari 3 kandidat — jika seimbang, kelereng yang disisihkan adalah jawabannya.

Complexity Thinking

$O(\log_3 n)$

Time complexity

$3^2 = 9 \geq 8 \checkmark$

kapasitas 2 penimbangan, terbukti cukup

$O(1)$ Space

Tanpa struktur data tambahan

SERABackend Technical Test 2026 · Section 1 Soal 1 · Bintang Wijaya · github.com/rhecustein/sera-test

Reasoning

Timbangan neraca dua lengan menghasilkan tepat 3 kemungkinan output setiap penimbangan: kiri lebih berat, kanan lebih berat, atau seimbang. Membagi kelereng menjadi 3 grup (bukan 2) memaksimalkan informasi yang diperoleh dari setiap penimbangan.

Kunci strategi ini: grup C tidak ikut ditimbang pada penimbangan pertama. Jika A dan B seimbang, berarti kelereng berat pasti ada di C — tanpa perlu menimbanginya sama sekali pada ronde pertama. Ini mereduksi kandidat dari 8 menjadi maksimal 3 dalam satu langkah.

Complexity Thinking

Aspek	Nilai	Penjelasan
Time complexity	$O(\log_3 n)$	Setiap penimbangan mereduksi kandidat menjadi 1/3
Kapasitas maksimal	3^n kelereng	n = jumlah penimbangan. 2 penimbangan = $3^2 = 9$ kelereng
Verifikasi	$8 < 9$	Dengan 2 penimbangan, 8 kelereng terbukti cukup
Space complexity	$O(1)$	Tidak ada struktur data tambahan yang digunakan
Worst case	2 penimbangan	Selalu selesai tepat 2 penimbangan, tidak kurang tidak lebih

Uji Coba Visual

Idle

012selesai

Tekan "Langkah berikutnya" untuk memulai penimbangan pertama.

Grup A
123

Grup B
456

Grup C
78

Reset

Mulai simulasi

Acak kelereng berat

Penimbangan ke
0/2

Kandidat tersisa
8

Complexity
 $O(\log_3 n)$

Percobaan 1

0 1 2 selesai

Kiri lebih berat — kelereng berat ada di grup A (1,2,3). Grup B dan C gugur.

Grup A
1 2 3

Grup B
4 5 6

Grup C
7 8

Reset

Langkah berikutnya

Acak kelereng berat

Penimbangan ke
1/2

Kandidat tersisa
3

Complexity
 $O(\log_3 n)$

Percobaan 2

0 1 2 selesai

Seimbang! Kelereng 3 (yang disisihkan) adalah yang terberat!

Grup A
1 2 3

Grup B
4 5 6

Grup C
7 8

Reset

Reset

Acak kelereng berat

Penimbangan ke
2/2

Kandidat tersisa
1

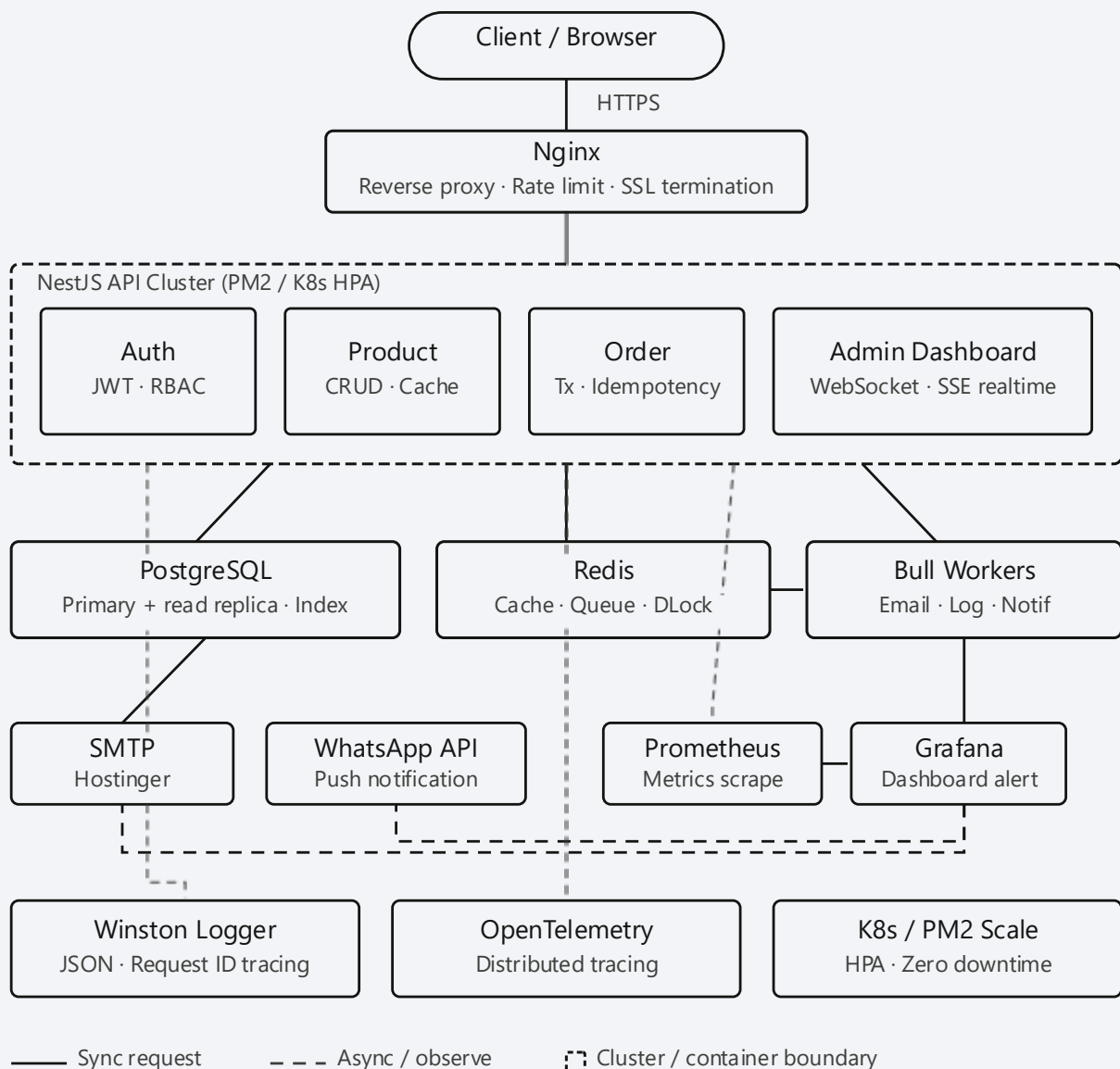
Complexity
 $O(\log_3 n)$

Soal 2 — System Design Thinking

- i Platform e-commerce dengan 10.000 transaksi per hari, proses pembayaran, email invoice, notifikasi WhatsApp, dan dashboard admin realtime.

Arsitektur Backend

Arsitektur yang dipilih adalah layered monolith dengan separation of concerns yang jelas, siap di-evolve ke microservices ketika traffic membutuhkan. Stack utama: NestJS + TypeScript sebagai API layer, PostgreSQL sebagai primary database, Redis untuk cache dan queue, dan Bull sebagai job processor.



E-commerce backend architecture — 10K transaksi/hari — SERA Technical Test 2026
Bintang Wijaya · NestJS + PostgreSQL + Redis + Bull Queue

Layer arsitektur dari atas ke bawah:

- Client (Web / Mobile / Admin Dashboard)
- Nginx — reverse proxy, SSL termination, rate limiting
- NestJS API Cluster — Auth, Product, Order, Admin services
- Data layer — PostgreSQL primary + read replica, Redis cache
- Async layer — Bull Queue workers (Email, Log, Notif)
- Monitoring — Prometheus + Grafana, Winston logger, OpenTelemetry

Cara Scaling

Komponen	Strategi Scaling
NestJS API	Horizontal scaling via PM2 cluster mode (dev) atau Kubernetes HPA (prod). Stateless by design, semua state di Redis/DB
PostgreSQL	Primary untuk write, read replica untuk query berat (dashboard admin, laporan). Connection pooling via PgBouncer
Redis	Cache product listing, user session, rate limit counter. Eviction policy: allkeys-lru
Bull Workers	Scale worker process secara independen dari API. Email worker bisa diberi 5 concurrency, log worker 10
Nginx	Upstream ke multiple API instances. Health check aktif — instance down otomatis di-remove

Mencegah Duplicate Event

1. Idempotency Key di setiap order request — UUID dibuat oleh client, dikirim sebagai header X-Idempotency-Key. Server simpan key di Redis (SETNX, TTL 24 jam) dan di kolom idempotency_key di tabel orders dengan UNIQUE constraint. Request kedua dengan key yang sama langsung return response pertama tanpa processing ulang.
2. Redis Distributed Lock saat proses payment — pakai SETNX dengan TTL pendek (30 detik) untuk pastikan satu transaksi hanya diproses satu worker sekalipun ada race condition.
3. Bull Queue job deduplication — job ID di-set menggunakan orderId sebagai prefix. Bull otomatis skip jika job dengan ID yang sama sudah ada di queue.
4. Database UNIQUE constraint pada idempotency_key sebagai safety net terakhir — kalau Redis lock bocor, DB masih mencegah duplikasi di level storage.

Logging & Monitoring

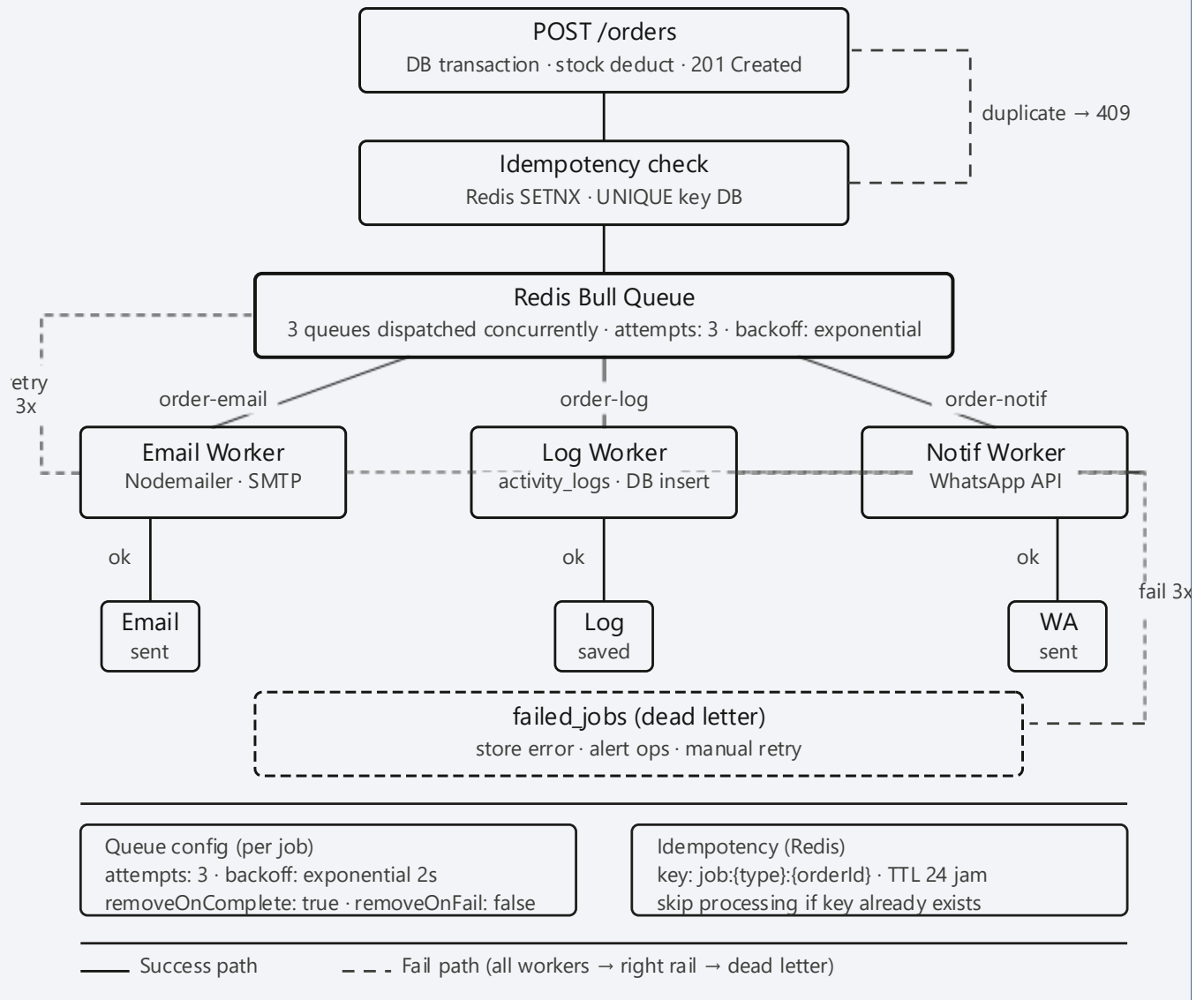
Structured logging menggunakan Winston dengan format JSON, propagasi Request ID di setiap log dalam satu request lifecycle:

```
{
  "timestamp": "2026-06-03T10:00:00.000Z",
  "level": "info",
  "requestId": "550e8400-e29b-41d4-a716-446655440000",
  "method": "POST",
  "path": "/orders",
  "statusCode": 201,
  "responseTime": "45ms",
  "userId": "user-uuid",
  "message": "Order created successfully"
}
```

Tool	Fungsi	Detail
Winston	Structured logging	JSON format, level: debug/info/warn/error, output ke file + console
OpenTelemetry	Distributed tracing	Trace request dari Nginx → NestJS → DB → Queue → Worker
Prometheus	Metrics scraping	Endpoint GET /metrics: total_requests, error_rate, response_time_ms
Grafana	Dashboard & alert	Visualisasi metric real-time, alert via Slack jika error rate > 5%

Kapan Menggunakan Queue / Message Broker

Situasi	Gunakan Queue	Alasan
Email invoice	Ya (selalu)	Tidak perlu response langsung, bisa retry jika SMTP down
WA notifikasi	Ya (selalu)	API WA bisa lambat/down, jangan block main thread
Activity log	Ya	Fire-and-forget, tidak perlu konfirmasi dari user
Bulk report	Ya	Operasi berat, bisa makan waktu menit ke jam
Stock deduction	Tidak	Harus atomik dengan order creation, pakai DB transaction
Auth/login	Tidak	User menunggu response, harus synchronous



Async queue flow — SERA Technical Test 2026 · Bintang Wijaya
NestJS + Bull Queue + Redis + PostgreSQL

Section 2 — Backend API Development

i Stack: NestJS + TypeScript. Demo: <https://sera.bintanx.com> · GitHub: github.com/rhecustein/sera-test · API Docs: <https://sera.bintanx.com/api/docs>

Authentication

Method	Endpoint	Auth	Deskripsi
POST	/auth/register	Public	Register user baru. Hash password bcrypt saltRounds:12. Return JWT
POST	/auth/login	Public	Login email+password. Validate bcrypt. Return JWT expires 1h

JWT Strategy: payload berisi { sub: userId, email, role }. Guard @JwtAuthGuard inject user ke request. Guard @RolesGuard cek role dari decorator @Roles("admin") atau @Roles("customer").

Product API

Method	Endpoint	Auth / Role	Deskripsi
GET	/products	Public	List produk + pagination (?page&limit) + search (?search) + sort (?sort&order)
GET	/products/:id	Public	Detail satu produk berdasarkan UUID
POST	/products	JWT + Admin	Buat produk baru. Validasi name, price>0, stock≥0
PATCH	/products/:id	JWT + Admin	Update parsial produk. Hanya field yang dikirim yang berubah
DELETE	/products/:id	JWT + Admin	Soft delete: set deleted_at. Tidak muncul di listing

Order API

Method	Endpoint	Auth / Role	Deskripsi
POST	/orders	JWT + Customer	Buat order. DB transaction: deduct stock + insert order + items. Dispatch 3 Bull jobs
GET	/orders	JWT	History order. Customer: milik sendiri. Admin: semua order + filter status
GET	/orders/:id	JWT	Detail order + items. Customer hanya bisa akses miliknya (403 jika bukan)

Standard Response Format

```
// Success
{
  "success": true,
  "statusCode": 200,
  "message": "Products fetched successfully",
  "data": { ... },
  "meta": { "page": 1, "limit": 10, "total": 100, "totalPages": 10 }
}

// Error
{
  "success": false,
  "statusCode": 400,
  "message": "Validation failed",
  "errors": [{ "field": "email", "message": "must be a valid email" }],
  "requestId": "uuid-v4",
  "timestamp": "2026-06-03T10:00:00.000Z"
}
```

Section 3 — Database & Optimization

Schema & Struktur Database

[**DIAGRAM: ERD — Entity Relationship Diagram (lihat /docs/ERD.png di repo)**]

Lihat file diagram di folder /docs repo GitHub

Tabel	Primary Key	Soft Delete	Keterangan
users	UUID	Ya (deleted_at)	name, email UNIQUE, password (bcrypt), role ENUM
products	UUID	Ya (deleted_at)	name, description, price DECIMAL(15,2), stock INT
orders	UUID	Ya (deleted_at)	user_id FK, total_price, status ENUM, idempotency_key UNIQUE
order_items	UUID	Tidak	order_id FK, product_id FK, quantity, price (snapshot harga)
activity_logs	UUID	Tidak	user_id FK, action, entity, entity_id, payload JSONB

Indexing Strategy

Tabel.Kolom	Tipe Index	Alasan
users.email	UNIQUE INDEX	Login lookup — query paling sering, harus O(1)
users.deleted_at	INDEX	Semua query filter WHERE deleted_at IS NULL
products.name	INDEX (GIN/trigram)	Full-text search dengan ILIKE — tanpa index = full table scan
products.deleted_at	INDEX	Listing selalu filter soft delete
orders.user_id	INDEX	History per user — join paling sering ke table orders
orders.idempotency_key	UNIQUE INDEX	Lookup untuk duplicate detection — harus unik dan cepat
order_items.order_id	INDEX	Eager load items per order — N+1 prevention

Alasan Keputusan Desain

- UUID sebagai PK (bukan auto-increment integer): distributed-safe, tidak bisa di-enumerate oleh attacker, kompatibel jika nanti di-shard.
- Price snapshot di order_items: harga produk bisa berubah kapan saja. Order harus preserve harga saat transaksi terjadi, bukan harga current.
- Soft delete via deleted_at: mempertahankan audit trail, tidak break foreign key reference, memungkinkan data recovery jika terjadi kesalahan.
- ENUM untuk role dan order status: validasi di level DB, bukan hanya di aplikasi. Mencegah data korup dari direct DB query.
- JSONB untuk activity_logs.payload: schema-flexible, bisa store arbitrary metadata per event tanpa ALTER TABLE.

Section 4 — Async Processing & Queue

i Tool: @nestjs/bull + bull + ioredis. Queue driver: Redis. 3 queue terpisah untuk isolasi failure.

Flow Async setelah Order Berhasil

[**DIAGRAM: Diagram Async Queue Flow — order-email, order-log, order-notif**]

Lihat file diagram di folder /docs repo GitHub

Setelah POST /orders berhasil (201 Created), Order Service dispatch 3 job secara concurrent ke Redis Bull Queue:

- order-email queue → SendInvoiceEmailJob: kirim HTML email invoice via SMTP Hostinger
- order-log queue → SaveActivityLogJob: insert row ke tabel activity_logs
- order-notif queue → SendWhatsAppNotificationJob: kirim notifikasi (placeholder / WA API)

Queue Configuration

```
// Config per queue (semua queue pakai setting ini)
{
  attempts: 3,                                // max retry
  backoff: {
    type: "exponential",
    delay: 2000                                // 2s, 4s, 8s
  },
  removeOnComplete: true,                      // hapus job sukses dari Redis
  removeOnFail: false                          // simpan job gagal untuk audit
}
```

Retry Mechanism

Bull otomatis retry job yang gagal menggunakan exponential backoff. Setiap attempt ke-n menunggu $2^n \times 2000\text{ms}$ sebelum dicoba ulang. Setelah 3 kali gagal, job masuk ke failed state dan dicatat di tabel failed_jobs (dead letter).

Dead Letter Handling

Job yang gagal setelah 3 attempt disimpan di Redis dengan status failed dan di-persist ke tabel failed_jobs di PostgreSQL. Data yang disimpan: job type, payload, error message, stack trace, attempt count, timestamps. Operator bisa trigger manual retry via admin endpoint atau Bull Board dashboard.

Idempotency Handling

```
// Di setiap processor, sebelum proses:
async processEmailJob(job: Job) {
  const key = `job:email:${job.data.orderId}`;
```

```
const exists = await this.redis.get(key);
if (exists) {
  this.logger.log(`[SKIP] Email already sent for order ${job.data.orderId}`);
  return; // idempotent - skip tanpa error
}
// proses email...
await this.redis.set(key, "1", "EX", 86400); // TTL 24 jam
}
```

Section 5 — Testing

i Framework: Jest + @nestjs/testing + Supertest. Target coverage: ≥70% (statements, lines, functions).

Testing Strategy

Layer	Tool	Yang Diuji
Unit Test	Jest + jest.fn()	Service logic: kalkulasi total, validasi stok, hash password, RBAC logic
Integration / E2E	Jest + Supertest	Full HTTP request/response: auth flow, CRUD products, create order, queue dispatch

Mocking Strategy

- TypeORM Repository: di-mock menggunakan createMockRepo() helper — semua method (find, save, delete, dll) adalah jest.fn() dengan return value yang dikontrol per test case.
- Bull Queue: mock getQueueToken() — method add() di-jest.fn() agar unit test tidak trigger real Redis.
- Nodemailer transporter: mock sendMail() — return { messageId: "mock-id" } untuk test email job tanpa SMTP connection.
- bcrypt: spy pada bcrypt.hash dan bcrypt.compare untuk kontrol hasil di edge case test.

Edge Cases yang Di-cover

Modul	Edge Cases
Auth	Register email duplikat → 409. Login password salah → 401. Token expired → 401. Token malformed → 401
Products	Create price=0 → 400. Update ID tidak ada → 404. Delete lalu akses → 404. RBAC customer create → 403
Orders	Quantity > stock → 400. product_id tidak ada → 404. Idempotency key duplikat → 409. Admin buat order → 403
Queue	Job gagal 3x → masuk failed_jobs. Job sama dikirim 2x → idempotency skip yang kedua

Menjalankan Test

```
# Unit test
npm run test

# E2E test
npm run test:e2e

# Coverage report
npm run test:cov
# Output: coverage/lcov-report/index.html
```


Section 6 — DevOps & Deployment

Docker Setup

Seluruh service dijalankan dalam Docker container. docker-compose.yml mendefinisikan 3 service utama + 1 Nginx:

Service	Port	Keterangan
app	3000	NestJS application. Multi-stage build — final image hanya production deps
postgres	5432	PostgreSQL 15-alpine. Data di-persist di named volume
redis	6379	Redis 7-alpine. Persistence AOF aktif untuk queue durability
nginx	80 / 443	Reverse proxy ke app:3000. SSL via Certbot. BONUS: rate limiting di level Nginx

Environment Separation

- .env — development local (tidak di-commit, ada di .gitignore)
- .env.example — template lengkap semua variable yang dibutuhkan
- .env.production — di-set langsung di server/CI sebagai environment variable (tidak ada file di repo)

Health Check Endpoint

```
// GET /health
{
  "status": "ok",
  "timestamp": "2026-06-03T10:00:00.000Z",
  "uptime": 3600,
  "database": "connected",
  "redis": "connected"
}

// Jika salah satu service down → HTTP 503
{ "status": "error", "database": "disconnected" }
```

Deployment di VPS

```
# Clone repo
git clone https://github.com/rhecustein/sera-test
cd sera-test

# Set environment variables
cp .env.example .env
# edit .env sesuai konfigurasi server

# Jalankan semua service
docker compose up -d

# Run database migration
docker compose exec app npm run migration:run
```

```
# Health check  
curl https://sera.bintanx.com/health
```

Bonus: Nginx Reverse Proxy + HTTPS

Nginx dikonfigurasi sebagai reverse proxy ke app:3000 dengan SSL certificate dari Let's Encrypt via Certbot. Domain aktif: sera.bintanx.com. HTTPS redirect aktif untuk semua HTTP request. Swagger UI tersedia di <https://sera.bintanx.com/api/docs>.

Section 7 — Observability

Structured Logging (Winston)

Semua log menggunakan format JSON dengan field standar. Request ID di-inject oleh middleware di awal setiap request dan di-propagate ke semua log dalam lifecycle request tersebut.

```
// Request middleware inject X-Request-ID
app.use((req, res, next) => {
  req.requestId = req.headers["x-request-id"] || uuidv4();
  res.setHeader("X-Request-ID", req.requestId);
  next();
});
```

Request Tracing

Setiap log line menyertakan requestId yang sama selama satu request lifecycle. Ini memungkinkan filtering semua log terkait satu request spesifik di log aggregator (Loki, Elasticsearch, atau CloudWatch).

Metrics Endpoint

```
// GET /metrics
{
  "total_requests": 1420,
  "total_errors": 12,
  "error_rate_percent": 0.85,
  "avg_response_time_ms": 87,
  "queue_pending": 3,
  "queue_failed": 0,
  "database_pool_active": 5,
  "uptime_seconds": 86400
}
```

Section 8 — Security

Keamanan	Implementasi	Detail
Rate Limiting	@nestjs/throttler	60 request/menit per IP. Kembalikan 429 jika melampaui
Password Hashing	bcrypt saltRounds:12	Password TIDAK pernah disimpan plain text. Tidak muncul di response
SQL Injection	TypeORM parameterized	Semua query pakai parameterized statement by default. Raw query dilarang
CORS	app.enableCors()	Whitelist origin dari CORS_ORIGIN env variable. Credential: true
Secure Headers	helmet middleware	X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, dll
Secret Management	.env + .gitignore	TIDAK ada secret hardcode di kode. .env.example sebagai template publik
JWT Expiry	expires: "1h"	Token expire setelah 1 jam. Refresh token flow dapat ditambahkan
Input Validation	class-validator	ValidationPipe global. Semua DTO punya validator decorator

Section 9 — Documentation

Setup Guide

Requirement: Node.js 20+, Docker, Docker Compose. Clone repo dan jalankan perintah berikut:

```
git clone https://github.com/rhecustein/sera-test
cd sera-test
cp .env.example .env           # isi semua variable
docker compose up -d          # start postgres + redis + app
npm run migration:run          # create tables
npm run seed                   # insert seeder (admin + customer)
open http://localhost:3000/api/docs # Swagger UI
```

API Documentation

Swagger UI tersedia di /api/docs pada kedua environment:

- Development: <http://localhost:3000/api/docs>
- Production: <https://sera.bintanx.com/api/docs>

Semua endpoint terdokumentasi lengkap dengan request body, response schema, dan authorization requirement. Swagger mendukung "Authorize" button untuk input JWT token secara langsung.

Struktur Repository

```
sera-test/
├── src/
│   ├── auth/           # Register, Login, JWT, Guards
│   ├── users/          # User entity & service
│   ├── products/       # CRUD Product + pagination + search
│   ├── orders/         # Create Order + history + detail
│   ├── queue/          # Bull processors: email, log, notif
│   ├── activity-logs/  # Activity log service
│   ├── common/         # Filters, interceptors, middleware
│   ├── config/         # DB, JWT, Redis, Mail config
│   ├── health/         # Health check endpoint
│   └── metrics/        # Metrics endpoint
├── test/               # E2E test specs
├── docs/
│   ├── ERD.png
│   ├── ARCHITECTURE.md
│   └── ASYNC_FLOW.md
├── docker-compose.yml
├── Dockerfile
├── .env.example
└── README.md
```

Seeder Default Credentials

Role	Email	Password	Akses
Admin	admin@sera.test	Admin123!	Semua endpoint termasuk CRUD product
Customer	customer@sera.test	Customer123!	Create order, lihat produk, history order milik sendiri

SERA Backend Technical Test — Jawaban Lengkap

Bintang Wijaya | bintangwijaya18@gmail.com | sera.bintanx.com | github.com/rhecustein/sera-test